

FLIGHT PRICE PREDICTION – MINI PROJECT DOCUMENTATION

Supervised Regression Machine Learning Project

1. Project Overview

The Flight Price Prediction project is a supervised regression machine learning project designed to predict flight ticket prices from flight-related features. The project follows a task-driven workflow from dataset understanding and data quality checks through exploratory data analysis, preprocessing, feature engineering, model building, evaluation, model saving, Flask deployment, and Docker containerization.

2. Problem Statement

Flight ticket prices vary based on factors such as airline, travel class, number of stops, duration, and the number of days left before departure. The objective is to build regression models that learn relationships between these features and the target variable price, then evaluate the models using standard regression metrics.

3. Dataset Selection & Understanding

Dataset Name: Flight Price Prediction Dataset

Dataset File: flightprice.csv

Records: 300,153

Columns: 12

Target Variable: price

Source: GitHub

The dataset is tab-separated. The source index column Unnamed: 0 is removed before modelling.

Dataset Link: [Open Flight Price Prediction Dataset](#)

4. Dataset Quality Check

The dataset quality-check workflow includes checking the shape, data types, missing values, duplicate rows, and source index column. The documentation records 0 missing values and 0 duplicate rows for the dataset.

5. Exploratory Data Analysis (EDA)

The following eight visualizations are required to understand the dataset and relationships with flight price:

1. Flight price distribution
2. Flight price box plot
3. Flights by airline
4. Price by class
5. Price by stops
6. Days left vs price
7. Duration vs price
8. Numeric correlation heatmap

6. Data Preprocessing

The preprocessing workflow includes duplicate removal, removal of the source index, missing-value imputers, conservative outlier treatment, One-Hot Encoding for categorical features, and StandardScaler for numerical features.

Pipeline and ColumnTransformer are used so that preprocessing is fitted on the training data and applied consistently to the test data, helping reduce data leakage.

7. Feature Engineering & Feature Selection

Categorical variables are transformed using One-Hot Encoding, while numerical variables are imputed and scaled. Feature selection is performed using SelectKBest with `f_regression`.

The high-cardinality flight feature is assessed as part of the feature-selection workflow.

8. Train-Test Split

The dataset is divided into training and testing sets using an 80:20 split with `random_state=42`. Preprocessing is fitted on the training data and then applied to the test data.

9. Model Building

The following regression models are required:

1. Linear Regression
2. Polynomial Regression
3. Ridge Regression
4. Lasso Regression

Polynomial regression uses a controlled numeric-feature workflow to manage polynomial expansion.

10. Model Evaluation

The models are evaluated using the following metrics:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Lower MAE and RMSE and higher R^2 indicate better measured performance. The actual metric values must be taken from the executed notebook and should not be fabricated.

11. Overfitting & Underfitting

The project considers the difference between underfitting and overfitting when evaluating the regression models. Model performance on training and test data is used to identify possible generalization issues.

12. Bias-Variance Consideration

Bias and variance are considered while comparing the models. Ridge and Lasso regression provide regularization that can help control model complexity.

13. Final Model Selection & Model Saving

The final model is selected after comparing the executed model results. The complete preprocessing and modelling pipeline is saved as `model.joblib` so that the same transformations can be used during prediction.

14. Flask Deployment

A Flask API is created in `app.py`. The API provides a POST `/predict` endpoint that accepts the required flight features and returns a predicted flight price.

15. Docker Containerization

The application is containerized using Docker. The required commands are:

```
docker build -t flight-price-api .  
docker run -p 5000:5000 flight-price-api
```

A screenshot of the running container and API response is included as part of the submission evidence.

16. Project Folder Structure

The project follows this structure:

Flight_Price_Prediction/
Flight_Price_Prediction.ipynb
flightprice.csv
app.py
model.joblib
requirements.txt
Dockerfile
README.md
Screenshots/

17. Mini Project Rule Compliance

The project covers the required workflow: Dataset → EDA → Preprocessing → Feature Engineering → Feature Selection → Model Building → Evaluation → Deployment.

The notebook, dataset, application, saved model, deployment files, README, and screenshots are included as the required project deliverables.

18. Conclusion

The Flight Price Prediction project demonstrates an end-to-end supervised regression workflow. The dataset is explored and prepared, relevant features are transformed and selected, multiple regression models are trained and evaluated, and the final pipeline is prepared for deployment using Flask and Docker.

The final model and performance metrics must be reported from the executed notebook results.

19. Final Submission Checklist

- Flight_Price_Prediction.ipynb
- flightprice.csv
- app.py
- model.joblib
- requirements.txt
- Dockerfile
- README.md
- Screenshots of EDA, model results, API response, and Docker execution
- Dataset link included in the documentation